Reviewer Pack — Coxeter Group Action Complexity and Communication Matrix Ran…

Entry fc756dd42100 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 13:27:30 UTC

Coxeter Group Action Complexity and Communication Matrix Rank Inequality

Entry ID: `fc756dd42100` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-03 13:27:30 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Geometry (Coxeter Groups) **Field B** (complexity object): Communication Complexity

Statement:

> For every instance of the MAX-CUT problem on n variables, the communication matrix rank k(n) is linearly related to the minimum number of moves required by a geodesic in the Coxeter group associated with the constraint graph G representing MAX-CUT. Specifically, k(n) = O(α(n)^2), where α(n) is the minimum number of moves for a geodesic.

Rationale (proposer's reasoning):

> Coxeter groups have been used to study symmetry and complexity in combinatorial problems. The action complexity of a Coxeter group on its associated graph can provide insights into the structure of the problem's constraint graph, which is closely related to communication complexity. A linear relationship between these two quantities could reveal a fundamental connection between geometric symmetry and network information flow.

Taxonomy category: `CoxeterGroupActionComplexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `2641461b4c973674`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all n ≤ 40 and across 30 random seeds, the communication matrix rank k(n) satisfies k(n) = O(α(n)^2), where α(n) is the minimum number of moves for a geodesic in the Coxeter group. The conjecture is falsified if there exists any instance where k(n) ≠ O(α(n)^2).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Coxeter group action" AND "communication complexity" AND "matrix rank inequality" - "MAX-CUT problem" AND geodesic AND "Coxeter group" AND communication - "geodesic moves in Coxeter group" AND "communication matrix rank" AND "linear relationship"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=0.0s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_max_cut_instance(n):
        edges = set()
        for i in range(n):
            for j in range(i + 1, n):
                if random.choice([True, False]):
                    edges.add((i, j))
        return edges

    def find_geodesic_length(edges, n):
        # Simplified version of finding the minimum number of moves
        # This is a placeholder and should be replaced with actual algorithm
        return n // 2

    def compute_communication_matrix_rank(edges, n):
        # Placeholder for communication matrix rank computation
        # This is a simplified version and should be replaced with actual algorithm
        return n * (n - 1) // 2

    n = random.randint(5, 40)
    edges = generate_max_cut_instance(n)

    alpha_n = find_geodesic_length(edges, n)
    k_n = compute_communication_matrix_rank(edges, n)
```

```
    if alpha_n == 0:
        return {
            "metric_name": "k(n)",
            "metric_value": float('inf'),
            "instances_tested": 1,
            "n_max": n,
            "conjecture_holds": False,
            "counterexample": "alpha_n is zero, division by zero"
        }

    if k_n > alpha_n ** 2:
        return {
            "metric_name": "k(n)",
            "metric_value": k_n,
            "instances_tested": 1,
            "n_max": n,
            "conjecture_holds": False,
            "counterexample": f"k(n) = {k_n}, alpha_n^2 = {alpha_n ** 2}"
        }

    return {
        "metric_name": "k(n)",
        "metric_value": k_n,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all(result["conjecture_holds"] for result in results):
        mean_value = sum(result["metric_value"] for result in results) / len(results)
        support_fraction = 1.0
        print(f"RESULT: SUPPORTED mean={mean_value} std=0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        counterexample = next(result["counterexample"] for result in results if not result["conjec
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
value': 210, 'instances_tested': 1, 'n_max': 21, 'conjecture_holds': False, 'counterexample': 'k(n) =
TRIAL: {'metric_name': 'k(n)', 'metric_value': 153, 'instances_tested': 1, 'n_max': 18, 'conjecture_h
TRIAL: {'metric_name': 'k(n)', 'metric_value': 105, 'instances_tested': 1, 'n_max': 15, 'conjecture_h
```

```
TRIAL: {'metric_name': 'k(n)', 'metric_value': 21, 'instances_tested': 1, 'n_max': 7, 'conjecture_hol
TRIAL: {'metric_name': 'k(n)', 'metric_value': 300, 'instances_tested': 1, 'n_max': 25, 'conjecture_h
TRIAL: {'metric_name': 'k(n)', 'metric_value': 253, 'instances_tested': 1, 'n_max': 23, 'conjecture_h
TRIAL: {'metric_name': 'k(n)', 'metric_value': 36, 'instances_tested': 1, 'n_max': 9, 'conjecture_hol
TRIAL: {'metric_name': 'k(n)', 'metric_value': 703, 'instances_tested': 1, 'n_max': 38, 'conjecture_h
TRIAL: {'metric_name': 'k(n)', 'metric_value': 120, 'instances_tested': 1, 'n_max': 16, 'conjecture_h
TRIAL: {'metric_name': 'k(n)', 'metric_value': 630, 'instances_tested': 1, 'n_max': 36, 'conjecture_h
TRIAL: {'metric_name': 'k(n)', 'metric_value': 171, 'instances_tested': 1, 'n_max': 19, 'conjecture_h
TRIAL: {'metric_name': 'k(n)', 'metric_value': 136, 'instances_tested': 1, 'n_max': 17, 'conjecture_h
RESULT: FALSIFIED counterexample="k(n) = 528, alpha_n^2 = 256" first_failing_seed=11
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> The test code uses placeholders for the geodesic length and communication matrix rank computation, which are crucial for evaluating the conjecture. Without actual implementations of these algorithms, the results cannot be reliably assessed.

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Safety rail: critic_challenge_falsified | original: The test results provide counterexamples where k(n) does not satisfy the inequality k(n) = O(α(n)^2), thus falsifying the conjecture. | next: Investigate and refine the algorithms for computing geodesic length and communication matrix rank to verify the conjecture or find a new inequality that holds true.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12017
2	propose	ollama_remote	glm4:latest	0	0	25832
3	preregistration	ollama_remote	glm4:latest	0	0	10602
4	novelty	ollama_remote	glm4:latest	0	0	8812
5	novelty	ollama_remote	glm4:latest	0	0	14259
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17940
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10601
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20166
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13194
10	critic	ollama_remote	glm4:latest	0	0	24073
11	judge	ollama_remote	glm4:latest	0	0	13434

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 170931 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/fc756dd42100.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/fc756dd42100.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/fc756dd42100.tar.gz` (if generated)